

Ansh Sharma

Roll No. 85

BE A Computer

Boston Housing Price Prediction using Deep Neural Networks

This notebook implements a deep neural network to predict housing prices using the Boston Housing dataset.

Problem Statement:

Linear regression by using Deep Neural network: Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.

Dataset Description

The Boston Housing dataset contains information about various features of houses in Boston suburbs and their prices. Each record has 13 features:

Feature	Description
CRIM	Per capita crime rate by town
ZN	Proportion of residential land zoned for lots over 25,000 sq.ft
INDUS	Proportion of non-retail business acres per town
CHAS	Charles River dummy variable (1 if tract bounds river; 0 otherwise)
NOX	Nitric oxides concentration (parts per 10 million)
RM	Average number of rooms per dwelling
AGE	Proportion of owner-occupied units built prior to 1940
DIS	Weighted distances to five Boston employment centers
RAD	Index of accessibility to radial highways
TAX	Full-value property-tax rate per \$10,000
PTRATIO	Pupil-teacher ratio by town
B	$1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town

Feature	Description
---------	-------------

LSTAT	% lower status of the population
-------	----------------------------------

Target Variable: MEDV (Median value of owner-occupied homes in \$1000s)

Import Required Libraries

```
In [1]: import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

# Set random seed for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

Data Loading and Preprocessing

We'll load the Boston Housing dataset and preprocess it by:

1. Loading from the original source
2. Normalizing features using StandardScaler
3. Splitting into train, validation, and test sets

```
In [2]: def load_boston_housing_data():
    """Load and preprocess the Boston Housing dataset from its original source
    data_url = "http://lib.stat.cmu.edu/datasets/boston"
    raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
    data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
    target = raw_df.values[1::2, 2]

    # Normalize features and target
    scaler_X = StandardScaler()
    scaler_y = StandardScaler()

    X = scaler_X.fit_transform(data)
    y = scaler_y.fit_transform(target.reshape(-1, 1))

    return X, y, scaler_y

# Load and preprocess data
X, y, scaler_y = load_boston_housing_data()

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2, random_state=42)
```

```
print(f"Training set shape: {X_train.shape}")
print(f"Validation set shape: {X_val.shape}")
print(f"Test set shape: {X_test.shape}")
```

Training set shape: (323, 13)
Validation set shape: (81, 13)
Test set shape: (102, 13)

Model Architecture

We'll create a deep neural network with the following features:

- Batch normalization for training stability
- Dropout layers for regularization
- L2 regularization on dense layers
- Multiple hidden layers with decreasing units

```
In [3]: def create_model():
        """Creates an improved neural network model for housing price prediction
        model = tf.keras.Sequential([
            # Input layer
            tf.keras.layers.Input(shape=(13,)),
            tf.keras.layers.BatchNormalization(),

            # First hidden layer
            tf.keras.layers.Dense(128, kernel_regularizer=tf.keras.regularizers.L2(0.01)),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Activation('relu'),
            tf.keras.layers.Dropout(0.3),

            # Second hidden layer
            tf.keras.layers.Dense(64, kernel_regularizer=tf.keras.regularizers.L2(0.01)),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Activation('relu'),
            tf.keras.layers.Dropout(0.2),

            # Third hidden layer
            tf.keras.layers.Dense(32, kernel_regularizer=tf.keras.regularizers.L2(0.01)),
            tf.keras.layers.BatchNormalization(),
            tf.keras.layers.Activation('relu'),
            tf.keras.layers.Dropout(0.1),

            # Output layer
            tf.keras.layers.Dense(1)
        ])

        return model

# Create and compile model
model = create_model()
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
```

```

    metrics=['mae']
)

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param
batch_normalization (BatchNormalization)	(None, 13)	
dense (Dense)	(None, 128)	1
batch_normalization_1 (BatchNormalization)	(None, 128)	
activation (Activation)	(None, 128)	
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 64)	8
batch_normalization_2 (BatchNormalization)	(None, 64)	
activation_1 (Activation)	(None, 64)	
dropout_1 (Dropout)	(None, 64)	
dense_2 (Dense)	(None, 32)	2
batch_normalization_3 (BatchNormalization)	(None, 32)	
activation_2 (Activation)	(None, 32)	
dropout_2 (Dropout)	(None, 32)	
dense_3 (Dense)	(None, 1)	

Total params: 13,109 (51.21 KB)

Trainable params: 12,635 (49.36 KB)

Non-trainable params: 474 (1.85 KB)

Model Training

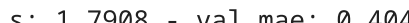
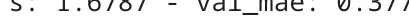
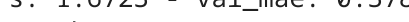
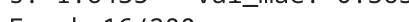
We'll train the model with:

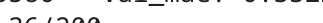
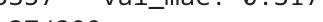
- Early stopping to prevent overfitting
- Learning rate reduction on plateau
- Batch size of 32
- Maximum 200 epochs

```
In [4]: # Create TensorFlow datasets
batch_size = 32
train_dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train)).shuffle(
val_dataset = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(batch
test_dataset = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(ba

# Callbacks
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=20,
        restore_best_weights=True,
        verbose=1
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.2,
        patience=10,
        min_lr=0.00001,
        verbose=1
    )
]

# Train the model
history = model.fit(
    train_dataset,
    epochs=200,
    validation_data=val_dataset,
    callbacks=callbacks,
    verbose=1
)
```

Epoch 1/200
11/11  **3s** 29ms/step - loss: 3.1960 - mae: 0.9936 - val_loss: 2.1415 - val_mae: 0.5745 - learning_rate: 0.0010
Epoch 2/200
11/11  **0s** 6ms/step - loss: 2.3969 - mae: 0.7262 - val_loss: 2.0091 - val_mae: 0.4895 - learning_rate: 0.0010
Epoch 3/200
11/11  **0s** 6ms/step - loss: 2.2680 - mae: 0.6547 - val_loss: 1.9687 - val_mae: 0.4882 - learning_rate: 0.0010
Epoch 4/200
11/11  **0s** 6ms/step - loss: 2.1208 - mae: 0.6026 - val_loss: 1.9584 - val_mae: 0.4955 - learning_rate: 0.0010
Epoch 5/200
11/11  **0s** 6ms/step - loss: 2.0749 - mae: 0.5831 - val_loss: 1.9126 - val_mae: 0.4739 - learning_rate: 0.0010
Epoch 6/200
11/11  **0s** 6ms/step - loss: 2.0347 - mae: 0.5900 - val_loss: 1.8501 - val_mae: 0.4306 - learning_rate: 0.0010
Epoch 7/200
11/11  **0s** 6ms/step - loss: 1.9803 - mae: 0.5599 - val_loss: 1.8086 - val_mae: 0.4067 - learning_rate: 0.0010
Epoch 8/200
11/11  **0s** 5ms/step - loss: 1.8943 - mae: 0.5262 - val_loss: 1.7908 - val_mae: 0.4041 - learning_rate: 0.0010
Epoch 9/200
11/11  **0s** 6ms/step - loss: 1.9317 - mae: 0.5423 - val_loss: 1.7564 - val_mae: 0.3880 - learning_rate: 0.0010
Epoch 10/200
11/11  **0s** 6ms/step - loss: 1.8327 - mae: 0.4987 - val_loss: 1.7306 - val_mae: 0.3839 - learning_rate: 0.0010
Epoch 11/200
11/11  **0s** 6ms/step - loss: 1.8388 - mae: 0.5116 - val_loss: 1.6995 - val_mae: 0.3708 - learning_rate: 0.0010
Epoch 12/200
11/11  **0s** 6ms/step - loss: 1.8051 - mae: 0.4862 - val_loss: 1.6749 - val_mae: 0.3605 - learning_rate: 0.0010
Epoch 13/200
11/11  **0s** 5ms/step - loss: 1.7097 - mae: 0.4546 - val_loss: 1.6787 - val_mae: 0.3772 - learning_rate: 0.0010
Epoch 14/200
11/11  **0s** 7ms/step - loss: 1.7142 - mae: 0.4711 - val_loss: 1.6723 - val_mae: 0.3786 - learning_rate: 0.0010
Epoch 15/200
11/11  **0s** 6ms/step - loss: 1.7922 - mae: 0.4979 - val_loss: 1.6435 - val_mae: 0.3637 - learning_rate: 0.0010
Epoch 16/200
11/11  **0s** 6ms/step - loss: 1.7015 - mae: 0.4752 - val_loss: 1.6208 - val_mae: 0.3574 - learning_rate: 0.0010
Epoch 17/200
11/11  **0s** 6ms/step - loss: 1.7061 - mae: 0.4890 - val_loss: 1.5971 - val_mae: 0.3479 - learning_rate: 0.0010
Epoch 18/200
11/11  **0s** 6ms/step - loss: 1.5937 - mae: 0.4275 - val_loss: 1.5785 - val_mae: 0.3432 - learning_rate: 0.0010
Epoch 19/200
11/11  **0s** 6ms/step - loss: 1.6672 - mae: 0.4572 - val_loss: 1.5785 - val_mae: 0.3432 - learning_rate: 0.0010

s: 1.5681 - val_mae: 0.3466 - learning_rate: 0.0010
Epoch 20/200
11/11  **0s** 6ms/step - loss: 1.7356 - mae: 0.5148 - val_loss: 1.5570 - val_mae: 0.3466 - learning_rate: 0.0010
Epoch 21/200
11/11  **0s** 5ms/step - loss: 1.6087 - mae: 0.4402 - val_loss: 1.5597 - val_mae: 0.3626 - learning_rate: 0.0010
Epoch 22/200
11/11  **0s** 6ms/step - loss: 1.5445 - mae: 0.4115 - val_loss: 1.5517 - val_mae: 0.3711 - learning_rate: 0.0010
Epoch 23/200
11/11  **0s** 6ms/step - loss: 1.5759 - mae: 0.4440 - val_loss: 1.5206 - val_mae: 0.3515 - learning_rate: 0.0010
Epoch 24/200
11/11  **0s** 5ms/step - loss: 1.5719 - mae: 0.4551 - val_loss: 1.5237 - val_mae: 0.3660 - learning_rate: 0.0010
Epoch 25/200
11/11  **0s** 6ms/step - loss: 1.6418 - mae: 0.4696 - val_loss: 1.4925 - val_mae: 0.3453 - learning_rate: 0.0010
Epoch 26/200
11/11  **0s** 6ms/step - loss: 1.5205 - mae: 0.4195 - val_loss: 1.4655 - val_mae: 0.3230 - learning_rate: 0.0010
Epoch 27/200
11/11  **0s** 6ms/step - loss: 1.4805 - mae: 0.4120 - val_loss: 1.4476 - val_mae: 0.3204 - learning_rate: 0.0010
Epoch 28/200
11/11  **0s** 7ms/step - loss: 1.5387 - mae: 0.4459 - val_loss: 1.4375 - val_mae: 0.3323 - learning_rate: 0.0010
Epoch 29/200
11/11  **0s** 7ms/step - loss: 1.5282 - mae: 0.4361 - val_loss: 1.4257 - val_mae: 0.3360 - learning_rate: 0.0010
Epoch 30/200
11/11  **0s** 6ms/step - loss: 1.5302 - mae: 0.4620 - val_loss: 1.4027 - val_mae: 0.3220 - learning_rate: 0.0010
Epoch 31/200
11/11  **0s** 6ms/step - loss: 1.4396 - mae: 0.4234 - val_loss: 1.3931 - val_mae: 0.3268 - learning_rate: 0.0010
Epoch 32/200
11/11  **0s** 6ms/step - loss: 1.4925 - mae: 0.4417 - val_loss: 1.3897 - val_mae: 0.3296 - learning_rate: 0.0010
Epoch 33/200
11/11  **0s** 5ms/step - loss: 1.3563 - mae: 0.3867 - val_loss: 1.3888 - val_mae: 0.3435 - learning_rate: 0.0010
Epoch 34/200
11/11  **0s** 6ms/step - loss: 1.4029 - mae: 0.4069 - val_loss: 1.3797 - val_mae: 0.3455 - learning_rate: 0.0010
Epoch 35/200
11/11  **0s** 7ms/step - loss: 1.4157 - mae: 0.4076 - val_loss: 1.3580 - val_mae: 0.3322 - learning_rate: 0.0010
Epoch 36/200
11/11  **0s** 6ms/step - loss: 1.3848 - mae: 0.4091 - val_loss: 1.3357 - val_mae: 0.3179 - learning_rate: 0.0010
Epoch 37/200
11/11  **0s** 6ms/step - loss: 1.3807 - mae: 0.4082 - val_loss: 1.3303 - val_mae: 0.3222 - learning_rate: 0.0010
Epoch 38/200

11/11 ————— 0s 6ms/step - loss: 1.3754 - mae: 0.4110 - val_loss: 1.3156 - val_mae: 0.3105 - learning_rate: 0.0010
Epoch 39/200

11/11 ————— 0s 6ms/step - loss: 1.4017 - mae: 0.4488 - val_loss: 1.3011 - val_mae: 0.3043 - learning_rate: 0.0010
Epoch 40/200

11/11 ————— 0s 5ms/step - loss: 1.4008 - mae: 0.4361 - val_loss: 1.2837 - val_mae: 0.3004 - learning_rate: 0.0010
Epoch 41/200

11/11 ————— 0s 6ms/step - loss: 1.2855 - mae: 0.3889 - val_loss: 1.2780 - val_mae: 0.3090 - learning_rate: 0.0010
Epoch 42/200

11/11 ————— 0s 6ms/step - loss: 1.3428 - mae: 0.4049 - val_loss: 1.2638 - val_mae: 0.3119 - learning_rate: 0.0010
Epoch 43/200

11/11 ————— 0s 5ms/step - loss: 1.3882 - mae: 0.4479 - val_loss: 1.2623 - val_mae: 0.3138 - learning_rate: 0.0010
Epoch 44/200

11/11 ————— 0s 6ms/step - loss: 1.4013 - mae: 0.4414 - val_loss: 1.2422 - val_mae: 0.2998 - learning_rate: 0.0010
Epoch 45/200

11/11 ————— 0s 7ms/step - loss: 1.2824 - mae: 0.3751 - val_loss: 1.2323 - val_mae: 0.2973 - learning_rate: 0.0010
Epoch 46/200

11/11 ————— 0s 5ms/step - loss: 1.3105 - mae: 0.4210 - val_loss: 1.2271 - val_mae: 0.3033 - learning_rate: 0.0010
Epoch 47/200

11/11 ————— 0s 6ms/step - loss: 1.2702 - mae: 0.3931 - val_loss: 1.2161 - val_mae: 0.3100 - learning_rate: 0.0010
Epoch 48/200

11/11 ————— 0s 6ms/step - loss: 1.2503 - mae: 0.4040 - val_loss: 1.1989 - val_mae: 0.3100 - learning_rate: 0.0010
Epoch 49/200

11/11 ————— 0s 6ms/step - loss: 1.2880 - mae: 0.4212 - val_loss: 1.1977 - val_mae: 0.3199 - learning_rate: 0.0010
Epoch 50/200

11/11 ————— 0s 5ms/step - loss: 1.2089 - mae: 0.3728 - val_loss: 1.1944 - val_mae: 0.3307 - learning_rate: 0.0010
Epoch 51/200

11/11 ————— 0s 6ms/step - loss: 1.2642 - mae: 0.4203 - val_loss: 1.1778 - val_mae: 0.3172 - learning_rate: 0.0010
Epoch 52/200

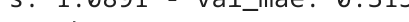
11/11 ————— 0s 6ms/step - loss: 1.2277 - mae: 0.3780 - val_loss: 1.1644 - val_mae: 0.3090 - learning_rate: 0.0010
Epoch 53/200

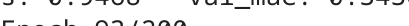
11/11 ————— 0s 6ms/step - loss: 1.2176 - mae: 0.3918 - val_loss: 1.1616 - val_mae: 0.3199 - learning_rate: 0.0010
Epoch 54/200

11/11 ————— 0s 6ms/step - loss: 1.2058 - mae: 0.3844 - val_loss: 1.1585 - val_mae: 0.3198 - learning_rate: 0.0010
Epoch 55/200

11/11 ————— 0s 6ms/step - loss: 1.2364 - mae: 0.4020 - val_loss: 1.1470 - val_mae: 0.3145 - learning_rate: 0.0010
Epoch 56/200

11/11 ————— 0s 6ms/step - loss: 1.1405 - mae: 0.3673 - val_loss: 1.1385 - val_mae: 0.3097 - learning_rate: 0.0010

Epoch 57/200
11/11  **0s** 6ms/step - loss: 1.1439 - mae: 0.3795 - val_loss: 1.1348 - val_mae: 0.3149 - learning_rate: 0.0010
Epoch 58/200
11/11  **0s** 6ms/step - loss: 1.1295 - mae: 0.3636 - val_loss: 1.1278 - val_mae: 0.3141 - learning_rate: 0.0010
Epoch 59/200
11/11  **0s** 5ms/step - loss: 1.2490 - mae: 0.4340 - val_loss: 1.1220 - val_mae: 0.3153 - learning_rate: 0.0010
Epoch 60/200
11/11  **0s** 6ms/step - loss: 1.1359 - mae: 0.3692 - val_loss: 1.1191 - val_mae: 0.3179 - learning_rate: 0.0010
Epoch 61/200
11/11  **0s** 6ms/step - loss: 1.1122 - mae: 0.3692 - val_loss: 1.1174 - val_mae: 0.3158 - learning_rate: 0.0010
Epoch 62/200
11/11  **0s** 5ms/step - loss: 1.1326 - mae: 0.3833 - val_loss: 1.1074 - val_mae: 0.3101 - learning_rate: 0.0010
Epoch 63/200
11/11  **0s** 6ms/step - loss: 1.1364 - mae: 0.3761 - val_loss: 1.0917 - val_mae: 0.3067 - learning_rate: 0.0010
Epoch 64/200
11/11  **0s** 5ms/step - loss: 1.1178 - mae: 0.3733 - val_loss: 1.0891 - val_mae: 0.3158 - learning_rate: 0.0010
Epoch 65/200
11/11  **0s** 6ms/step - loss: 1.0780 - mae: 0.3500 - val_loss: 1.0793 - val_mae: 0.3035 - learning_rate: 0.0010
Epoch 66/200
11/11  **0s** 6ms/step - loss: 1.1453 - mae: 0.3871 - val_loss: 1.0672 - val_mae: 0.2996 - learning_rate: 0.0010
Epoch 67/200
11/11  **0s** 6ms/step - loss: 1.1213 - mae: 0.3806 - val_loss: 1.0621 - val_mae: 0.2988 - learning_rate: 0.0010
Epoch 68/200
11/11  **0s** 6ms/step - loss: 1.1059 - mae: 0.3818 - val_loss: 1.0557 - val_mae: 0.2959 - learning_rate: 0.0010
Epoch 69/200
11/11  **0s** 6ms/step - loss: 1.1068 - mae: 0.3824 - val_loss: 1.0451 - val_mae: 0.3001 - learning_rate: 0.0010
Epoch 70/200
11/11  **0s** 6ms/step - loss: 1.1003 - mae: 0.3663 - val_loss: 1.0411 - val_mae: 0.3075 - learning_rate: 0.0010
Epoch 71/200
11/11  **0s** 5ms/step - loss: 1.1085 - mae: 0.3857 - val_loss: 1.0467 - val_mae: 0.3203 - learning_rate: 0.0010
Epoch 72/200
11/11  **0s** 6ms/step - loss: 1.0950 - mae: 0.3941 - val_loss: 1.0401 - val_mae: 0.3212 - learning_rate: 0.0010
Epoch 73/200
11/11  **0s** 6ms/step - loss: 1.0731 - mae: 0.3663 - val_loss: 1.0389 - val_mae: 0.3207 - learning_rate: 0.0010
Epoch 74/200
11/11  **0s** 5ms/step - loss: 1.0419 - mae: 0.3481 - val_loss: 1.0306 - val_mae: 0.3184 - learning_rate: 0.0010
Epoch 75/200
11/11  **0s** 6ms/step - loss: 1.0368 - mae: 0.3613 - val_loss:

s: 1.0159 - val_mae: 0.3131 - learning_rate: 0.0010
Epoch 76/200
11/11  **0s** 5ms/step - loss: 1.0756 - mae: 0.3894 - val_loss: 1.0108 - val_mae: 0.3150 - learning_rate: 0.0010
Epoch 77/200
11/11  **0s** 6ms/step - loss: 0.9947 - mae: 0.3346 - val_loss: 1.0057 - val_mae: 0.3168 - learning_rate: 0.0010
Epoch 78/200
11/11  **0s** 6ms/step - loss: 1.0132 - mae: 0.3647 - val_loss: 0.9961 - val_mae: 0.3192 - learning_rate: 0.0010
Epoch 79/200
11/11  **0s** 6ms/step - loss: 0.9769 - mae: 0.3489 - val_loss: 0.9845 - val_mae: 0.3200 - learning_rate: 0.0010
Epoch 80/200
11/11  **0s** 6ms/step - loss: 0.9953 - mae: 0.3627 - val_loss: 0.9746 - val_mae: 0.3171 - learning_rate: 0.0010
Epoch 81/200
11/11  **0s** 5ms/step - loss: 0.9942 - mae: 0.3713 - val_loss: 0.9678 - val_mae: 0.3130 - learning_rate: 0.0010
Epoch 82/200
11/11  **0s** 5ms/step - loss: 0.9730 - mae: 0.3525 - val_loss: 0.9617 - val_mae: 0.3115 - learning_rate: 0.0010
Epoch 83/200
11/11  **0s** 5ms/step - loss: 0.9185 - mae: 0.3272 - val_loss: 0.9576 - val_mae: 0.3118 - learning_rate: 0.0010
Epoch 84/200
11/11  **0s** 5ms/step - loss: 0.9863 - mae: 0.3718 - val_loss: 0.9659 - val_mae: 0.3251 - learning_rate: 0.0010
Epoch 85/200
11/11  **0s** 5ms/step - loss: 0.9329 - mae: 0.3410 - val_loss: 0.9732 - val_mae: 0.3322 - learning_rate: 0.0010
Epoch 86/200
11/11  **0s** 6ms/step - loss: 0.9612 - mae: 0.3618 - val_loss: 0.9559 - val_mae: 0.3208 - learning_rate: 0.0010
Epoch 87/200
11/11  **0s** 5ms/step - loss: 0.9403 - mae: 0.3501 - val_loss: 0.9514 - val_mae: 0.3110 - learning_rate: 0.0010
Epoch 88/200
11/11  **0s** 6ms/step - loss: 0.9208 - mae: 0.3324 - val_loss: 0.9377 - val_mae: 0.3033 - learning_rate: 0.0010
Epoch 89/200
11/11  **0s** 5ms/step - loss: 0.9059 - mae: 0.3276 - val_loss: 0.9343 - val_mae: 0.3125 - learning_rate: 0.0010
Epoch 90/200
11/11  **0s** 5ms/step - loss: 0.9478 - mae: 0.3604 - val_loss: 0.9328 - val_mae: 0.3171 - learning_rate: 0.0010
Epoch 91/200
11/11  **0s** 7ms/step - loss: 0.9600 - mae: 0.3677 - val_loss: 0.9410 - val_mae: 0.3242 - learning_rate: 0.0010
Epoch 92/200
11/11  **0s** 5ms/step - loss: 0.9126 - mae: 0.3373 - val_loss: 0.9468 - val_mae: 0.3430 - learning_rate: 0.0010
Epoch 93/200
11/11  **0s** 5ms/step - loss: 0.8901 - mae: 0.3365 - val_loss: 0.9381 - val_mae: 0.3361 - learning_rate: 0.0010
Epoch 94/200

11/11  0s 5ms/step - loss: 0.9455 - mae: 0.3652 - val_loss: 0.9197 - val_mae: 0.3268 - learning_rate: 0.0010
Epoch 95/200

11/11  0s 5ms/step - loss: 0.9611 - mae: 0.3838 - val_loss: 0.8957 - val_mae: 0.3106 - learning_rate: 0.0010
Epoch 96/200

11/11  0s 6ms/step - loss: 0.8964 - mae: 0.3574 - val_loss: 0.8941 - val_mae: 0.3156 - learning_rate: 0.0010
Epoch 97/200

11/11  0s 5ms/step - loss: 0.8807 - mae: 0.3353 - val_loss: 0.9036 - val_mae: 0.3341 - learning_rate: 0.0010
Epoch 98/200

11/11  0s 4ms/step - loss: 0.8548 - mae: 0.3239 - val_loss: 0.9008 - val_mae: 0.3342 - learning_rate: 0.0010
Epoch 99/200

11/11  0s 5ms/step - loss: 0.8770 - mae: 0.3457 - val_loss: 0.8942 - val_mae: 0.3309 - learning_rate: 0.0010
Epoch 100/200

11/11  0s 5ms/step - loss: 0.9112 - mae: 0.3653 - val_loss: 0.8843 - val_mae: 0.3328 - learning_rate: 0.0010
Epoch 101/200

11/11  0s 5ms/step - loss: 0.8584 - mae: 0.3446 - val_loss: 0.8746 - val_mae: 0.3348 - learning_rate: 0.0010
Epoch 102/200

11/11  0s 7ms/step - loss: 0.8712 - mae: 0.3463 - val_loss: 0.8697 - val_mae: 0.3400 - learning_rate: 0.0010
Epoch 103/200

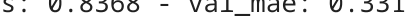
11/11  0s 5ms/step - loss: 0.8294 - mae: 0.3299 - val_loss: 0.8708 - val_mae: 0.3434 - learning_rate: 0.0010
Epoch 104/200

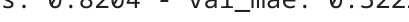
11/11  0s 5ms/step - loss: 0.8387 - mae: 0.3271 - val_loss: 0.8488 - val_mae: 0.3250 - learning_rate: 0.0010
Epoch 105/200

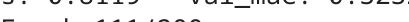
11/11  0s 6ms/step - loss: 0.8574 - mae: 0.3543 - val_loss: 0.8295 - val_mae: 0.3082 - learning_rate: 0.0010
Epoch 106/200

11/11  0s 5ms/step - loss: 0.8635 - mae: 0.3592 - val_loss: 0.8284 - val_mae: 0.3157 - learning_rate: 0.0010
Epoch 107/200

11/11  0s 5ms/step - loss: 0.8575 - mae: 0.3586 - val_loss: 0.8347 - val_mae: 0.3270 - learning_rate: 0.0010
Epoch 108/200

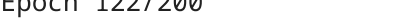
11/11  0s 5ms/step - loss: 0.8778 - mae: 0.3674 - val_loss: 0.8368 - val_mae: 0.3311 - learning_rate: 0.0010
Epoch 109/200

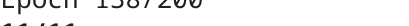
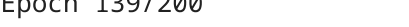
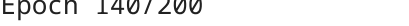
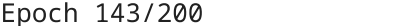
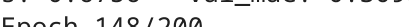
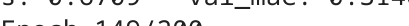
11/11  0s 5ms/step - loss: 0.8055 - mae: 0.3234 - val_loss: 0.8204 - val_mae: 0.3222 - learning_rate: 0.0010
Epoch 110/200

11/11  0s 5ms/step - loss: 0.8177 - mae: 0.3418 - val_loss: 0.8119 - val_mae: 0.3232 - learning_rate: 0.0010
Epoch 111/200

11/11  0s 5ms/step - loss: 0.7797 - mae: 0.3263 - val_loss: 0.8003 - val_mae: 0.3102 - learning_rate: 0.0010
Epoch 112/200

11/11  0s 5ms/step - loss: 0.8294 - mae: 0.3506 - val_loss: 0.7962 - val_mae: 0.3095 - learning_rate: 0.0010

Epoch 113/200
11/11  0s 5ms/step - loss: 0.8148 - mae: 0.3576 - val_loss: 0.8021 - val_mae: 0.3195 - learning_rate: 0.0010
Epoch 114/200
11/11  0s 5ms/step - loss: 0.7850 - mae: 0.3313 - val_loss: 0.8065 - val_mae: 0.3309 - learning_rate: 0.0010
Epoch 115/200
11/11  0s 5ms/step - loss: 0.8413 - mae: 0.3793 - val_loss: 0.8016 - val_mae: 0.3324 - learning_rate: 0.0010
Epoch 116/200
11/11  0s 5ms/step - loss: 0.7544 - mae: 0.3224 - val_loss: 0.7910 - val_mae: 0.3312 - learning_rate: 0.0010
Epoch 117/200
11/11  0s 5ms/step - loss: 0.7735 - mae: 0.3370 - val_loss: 0.7848 - val_mae: 0.3340 - learning_rate: 0.0010
Epoch 118/200
11/11  0s 6ms/step - loss: 0.7564 - mae: 0.3196 - val_loss: 0.7784 - val_mae: 0.3336 - learning_rate: 0.0010
Epoch 119/200
11/11  0s 5ms/step - loss: 0.8154 - mae: 0.3630 - val_loss: 0.7603 - val_mae: 0.3156 - learning_rate: 0.0010
Epoch 120/200
11/11  0s 6ms/step - loss: 0.7713 - mae: 0.3383 - val_loss: 0.7615 - val_mae: 0.3166 - learning_rate: 0.0010
Epoch 121/200
11/11  0s 6ms/step - loss: 0.8142 - mae: 0.3635 - val_loss: 0.7589 - val_mae: 0.3109 - learning_rate: 0.0010
Epoch 122/200
11/11  0s 6ms/step - loss: 0.7739 - mae: 0.3586 - val_loss: 0.7568 - val_mae: 0.3102 - learning_rate: 0.0010
Epoch 123/200
11/11  0s 5ms/step - loss: 0.7564 - mae: 0.3548 - val_loss: 0.7511 - val_mae: 0.3069 - learning_rate: 0.0010
Epoch 124/200
11/11  0s 5ms/step - loss: 0.7380 - mae: 0.3308 - val_loss: 0.7388 - val_mae: 0.3083 - learning_rate: 0.0010
Epoch 125/200
11/11  0s 6ms/step - loss: 0.7579 - mae: 0.3511 - val_loss: 0.7289 - val_mae: 0.3064 - learning_rate: 0.0010
Epoch 126/200
11/11  0s 5ms/step - loss: 0.7150 - mae: 0.3266 - val_loss: 0.7272 - val_mae: 0.3112 - learning_rate: 0.0010
Epoch 127/200
11/11  0s 5ms/step - loss: 0.7448 - mae: 0.3484 - val_loss: 0.7291 - val_mae: 0.3151 - learning_rate: 0.0010
Epoch 128/200
11/11  0s 5ms/step - loss: 0.7526 - mae: 0.3520 - val_loss: 0.7384 - val_mae: 0.3255 - learning_rate: 0.0010
Epoch 129/200
11/11  0s 5ms/step - loss: 0.7631 - mae: 0.3477 - val_loss: 0.7280 - val_mae: 0.3185 - learning_rate: 0.0010
Epoch 130/200
11/11  0s 5ms/step - loss: 0.6903 - mae: 0.3233 - val_loss: 0.7493 - val_mae: 0.3472 - learning_rate: 0.0010
Epoch 131/200
11/11  0s 5ms/step - loss: 0.7138 - mae: 0.3388 - val_loss:

s: 0.7423 - val_mae: 0.3445 - learning_rate: 0.0010
Epoch 132/200
11/11  **0s** 5ms/step - loss: 0.7177 - mae: 0.3410 - val_loss: 0.7118 - val_mae: 0.3138 - learning_rate: 0.0010
Epoch 133/200
11/11  **0s** 5ms/step - loss: 0.7254 - mae: 0.3687 - val_loss: 0.7017 - val_mae: 0.3066 - learning_rate: 0.0010
Epoch 134/200
11/11  **0s** 5ms/step - loss: 0.7425 - mae: 0.3589 - val_loss: 0.6948 - val_mae: 0.3054 - learning_rate: 0.0010
Epoch 135/200
11/11  **0s** 5ms/step - loss: 0.7157 - mae: 0.3437 - val_loss: 0.6899 - val_mae: 0.3068 - learning_rate: 0.0010
Epoch 136/200
11/11  **0s** 5ms/step - loss: 0.7365 - mae: 0.3776 - val_loss: 0.7011 - val_mae: 0.3162 - learning_rate: 0.0010
Epoch 137/200
11/11  **0s** 5ms/step - loss: 0.6954 - mae: 0.3345 - val_loss: 0.7054 - val_mae: 0.3236 - learning_rate: 0.0010
Epoch 138/200
11/11  **0s** 5ms/step - loss: 0.6764 - mae: 0.3398 - val_loss: 0.7129 - val_mae: 0.3335 - learning_rate: 0.0010
Epoch 139/200
11/11  **0s** 5ms/step - loss: 0.7278 - mae: 0.3601 - val_loss: 0.7132 - val_mae: 0.3422 - learning_rate: 0.0010
Epoch 140/200
11/11  **0s** 5ms/step - loss: 0.6528 - mae: 0.3216 - val_loss: 0.6945 - val_mae: 0.3238 - learning_rate: 0.0010
Epoch 141/200
11/11  **0s** 5ms/step - loss: 0.6833 - mae: 0.3525 - val_loss: 0.6981 - val_mae: 0.3240 - learning_rate: 0.0010
Epoch 142/200
11/11  **0s** 5ms/step - loss: 0.6821 - mae: 0.3469 - val_loss: 0.6787 - val_mae: 0.3125 - learning_rate: 0.0010
Epoch 143/200
11/11  **0s** 5ms/step - loss: 0.6997 - mae: 0.3672 - val_loss: 0.6792 - val_mae: 0.3289 - learning_rate: 0.0010
Epoch 144/200
11/11  **0s** 5ms/step - loss: 0.6470 - mae: 0.3188 - val_loss: 0.6900 - val_mae: 0.3372 - learning_rate: 0.0010
Epoch 145/200
11/11  **0s** 5ms/step - loss: 0.6812 - mae: 0.3552 - val_loss: 0.6779 - val_mae: 0.3138 - learning_rate: 0.0010
Epoch 146/200
11/11  **0s** 5ms/step - loss: 0.6577 - mae: 0.3369 - val_loss: 0.6866 - val_mae: 0.3183 - learning_rate: 0.0010
Epoch 147/200
11/11  **0s** 5ms/step - loss: 0.6214 - mae: 0.3039 - val_loss: 0.6756 - val_mae: 0.3098 - learning_rate: 0.0010
Epoch 148/200
11/11  **0s** 5ms/step - loss: 0.6422 - mae: 0.3270 - val_loss: 0.6709 - val_mae: 0.3146 - learning_rate: 0.0010
Epoch 149/200
11/11  **0s** 5ms/step - loss: 0.6491 - mae: 0.3318 - val_loss: 0.6833 - val_mae: 0.3317 - learning_rate: 0.0010
Epoch 150/200

11/11 ————— 0s 5ms/step - loss: 0.6174 - mae: 0.3097 - val_loss: 0.6925 - val_mae: 0.3418 - learning_rate: 0.0010
Epoch 151/200

11/11 ————— 0s 4ms/step - loss: 0.6586 - mae: 0.3435 - val_loss: 0.6740 - val_mae: 0.3339 - learning_rate: 0.0010
Epoch 152/200

11/11 ————— 0s 5ms/step - loss: 0.6353 - mae: 0.3190 - val_loss: 0.6584 - val_mae: 0.3261 - learning_rate: 0.0010
Epoch 153/200

11/11 ————— 0s 5ms/step - loss: 0.7105 - mae: 0.3764 - val_loss: 0.6540 - val_mae: 0.3231 - learning_rate: 0.0010
Epoch 154/200

11/11 ————— 0s 5ms/step - loss: 0.6150 - mae: 0.3178 - val_loss: 0.6373 - val_mae: 0.3040 - learning_rate: 0.0010
Epoch 155/200

11/11 ————— 0s 5ms/step - loss: 0.6353 - mae: 0.3388 - val_loss: 0.6343 - val_mae: 0.3033 - learning_rate: 0.0010
Epoch 156/200

11/11 ————— 0s 5ms/step - loss: 0.5725 - mae: 0.3025 - val_loss: 0.6389 - val_mae: 0.3119 - learning_rate: 0.0010
Epoch 157/200

11/11 ————— 0s 5ms/step - loss: 0.5885 - mae: 0.3120 - val_loss: 0.6290 - val_mae: 0.3086 - learning_rate: 0.0010
Epoch 158/200

11/11 ————— 0s 5ms/step - loss: 0.6053 - mae: 0.3155 - val_loss: 0.6204 - val_mae: 0.3169 - learning_rate: 0.0010
Epoch 159/200

11/11 ————— 0s 5ms/step - loss: 0.6296 - mae: 0.3371 - val_loss: 0.6264 - val_mae: 0.3253 - learning_rate: 0.0010
Epoch 160/200

11/11 ————— 0s 5ms/step - loss: 0.6151 - mae: 0.3402 - val_loss: 0.6358 - val_mae: 0.3390 - learning_rate: 0.0010
Epoch 161/200

11/11 ————— 0s 5ms/step - loss: 0.5731 - mae: 0.3019 - val_loss: 0.6329 - val_mae: 0.3396 - learning_rate: 0.0010
Epoch 162/200

11/11 ————— 0s 5ms/step - loss: 0.5705 - mae: 0.3128 - val_loss: 0.6176 - val_mae: 0.3304 - learning_rate: 0.0010
Epoch 163/200

11/11 ————— 0s 5ms/step - loss: 0.5998 - mae: 0.3335 - val_loss: 0.6131 - val_mae: 0.3261 - learning_rate: 0.0010
Epoch 164/200

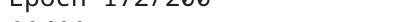
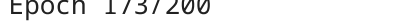
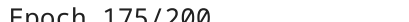
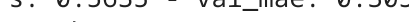
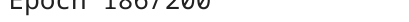
11/11 ————— 0s 5ms/step - loss: 0.6562 - mae: 0.3832 - val_loss: 0.6197 - val_mae: 0.3230 - learning_rate: 0.0010
Epoch 165/200

11/11 ————— 0s 5ms/step - loss: 0.5956 - mae: 0.3184 - val_loss: 0.6213 - val_mae: 0.3293 - learning_rate: 0.0010
Epoch 166/200

11/11 ————— 0s 5ms/step - loss: 0.5697 - mae: 0.3067 - val_loss: 0.6151 - val_mae: 0.3265 - learning_rate: 0.0010
Epoch 167/200

11/11 ————— 0s 5ms/step - loss: 0.6154 - mae: 0.3403 - val_loss: 0.6063 - val_mae: 0.3219 - learning_rate: 0.0010
Epoch 168/200

11/11 ————— 0s 5ms/step - loss: 0.6486 - mae: 0.3768 - val_loss: 0.6033 - val_mae: 0.3191 - learning_rate: 0.0010

Epoch 169/200
11/11  **0s** 5ms/step - loss: 0.5593 - mae: 0.3069 - val_loss: 0.5936 - val_mae: 0.3100 - learning_rate: 0.0010
Epoch 170/200
11/11  **0s** 4ms/step - loss: 0.5539 - mae: 0.3170 - val_loss: 0.5975 - val_mae: 0.3145 - learning_rate: 0.0010
Epoch 171/200
11/11  **0s** 5ms/step - loss: 0.5797 - mae: 0.3286 - val_loss: 0.5897 - val_mae: 0.3089 - learning_rate: 0.0010
Epoch 172/200
11/11  **0s** 5ms/step - loss: 0.5859 - mae: 0.3440 - val_loss: 0.5952 - val_mae: 0.3137 - learning_rate: 0.0010
Epoch 173/200
11/11  **0s** 5ms/step - loss: 0.5899 - mae: 0.3347 - val_loss: 0.6013 - val_mae: 0.3336 - learning_rate: 0.0010
Epoch 174/200
11/11  **0s** 5ms/step - loss: 0.6007 - mae: 0.3478 - val_loss: 0.5794 - val_mae: 0.3171 - learning_rate: 0.0010
Epoch 175/200
11/11  **0s** 5ms/step - loss: 0.5791 - mae: 0.3397 - val_loss: 0.5796 - val_mae: 0.3132 - learning_rate: 0.0010
Epoch 176/200
11/11  **0s** 6ms/step - loss: 0.5600 - mae: 0.3238 - val_loss: 0.5635 - val_mae: 0.3051 - learning_rate: 0.0010
Epoch 177/200
11/11  **0s** 5ms/step - loss: 0.5731 - mae: 0.3355 - val_loss: 0.5568 - val_mae: 0.3116 - learning_rate: 0.0010
Epoch 178/200
11/11  **0s** 5ms/step - loss: 0.5191 - mae: 0.3015 - val_loss: 0.5596 - val_mae: 0.3178 - learning_rate: 0.0010
Epoch 179/200
11/11  **0s** 4ms/step - loss: 0.5530 - mae: 0.3148 - val_loss: 0.5901 - val_mae: 0.3480 - learning_rate: 0.0010
Epoch 180/200
11/11  **0s** 5ms/step - loss: 0.5543 - mae: 0.3404 - val_loss: 0.5669 - val_mae: 0.3238 - learning_rate: 0.0010
Epoch 181/200
11/11  **0s** 5ms/step - loss: 0.5573 - mae: 0.3351 - val_loss: 0.5427 - val_mae: 0.2939 - learning_rate: 0.0010
Epoch 182/200
11/11  **0s** 5ms/step - loss: 0.5379 - mae: 0.3162 - val_loss: 0.5598 - val_mae: 0.3180 - learning_rate: 0.0010
Epoch 183/200
11/11  **0s** 5ms/step - loss: 0.5901 - mae: 0.3606 - val_loss: 0.5673 - val_mae: 0.3300 - learning_rate: 0.0010
Epoch 184/200
11/11  **0s** 5ms/step - loss: 0.5645 - mae: 0.3399 - val_loss: 0.5807 - val_mae: 0.3495 - learning_rate: 0.0010
Epoch 185/200
11/11  **0s** 4ms/step - loss: 0.5140 - mae: 0.3108 - val_loss: 0.5545 - val_mae: 0.3278 - learning_rate: 0.0010
Epoch 186/200
11/11  **0s** 5ms/step - loss: 0.5369 - mae: 0.3259 - val_loss: 0.5391 - val_mae: 0.3133 - learning_rate: 0.0010
Epoch 187/200
11/11  **0s** 4ms/step - loss: 0.5322 - mae: 0.3285 - val_loss:


```

s: 0.5499 - val_mae: 0.3229 - learning_rate: 0.0010
Epoch 188/200
11/11  0s 5ms/step - loss: 0.5333 - mae: 0.3288 - val_loss: 0.5659 - val_mae: 0.3331 - learning_rate: 0.0010
Epoch 189/200
11/11  0s 5ms/step - loss: 0.5345 - mae: 0.3185 - val_loss: 0.5585 - val_mae: 0.3295 - learning_rate: 0.0010
Epoch 190/200
11/11  0s 5ms/step - loss: 0.5202 - mae: 0.3268 - val_loss: 0.5329 - val_mae: 0.3242 - learning_rate: 0.0010
Epoch 191/200
11/11  0s 5ms/step - loss: 0.5336 - mae: 0.3397 - val_loss: 0.5217 - val_mae: 0.3232 - learning_rate: 0.0010
Epoch 192/200
11/11  0s 5ms/step - loss: 0.5174 - mae: 0.3367 - val_loss: 0.5066 - val_mae: 0.3065 - learning_rate: 0.0010
Epoch 193/200
11/11  0s 5ms/step - loss: 0.4966 - mae: 0.3099 - val_loss: 0.5159 - val_mae: 0.3027 - learning_rate: 0.0010
Epoch 194/200
11/11  0s 5ms/step - loss: 0.6035 - mae: 0.3993 - val_loss: 0.5237 - val_mae: 0.3075 - learning_rate: 0.0010
Epoch 195/200
11/11  0s 5ms/step - loss: 0.5402 - mae: 0.3367 - val_loss: 0.5264 - val_mae: 0.3215 - learning_rate: 0.0010
Epoch 196/200
11/11  0s 4ms/step - loss: 0.5120 - mae: 0.3256 - val_loss: 0.5373 - val_mae: 0.3379 - learning_rate: 0.0010
Epoch 197/200
11/11  0s 4ms/step - loss: 0.4954 - mae: 0.3094 - val_loss: 0.5170 - val_mae: 0.3211 - learning_rate: 0.0010
Epoch 198/200
11/11  0s 5ms/step - loss: 0.4610 - mae: 0.2842 - val_loss: 0.4957 - val_mae: 0.3040 - learning_rate: 0.0010
Epoch 199/200
11/11  0s 5ms/step - loss: 0.4602 - mae: 0.2873 - val_loss: 0.4898 - val_mae: 0.2993 - learning_rate: 0.0010
Epoch 200/200
11/11  0s 5ms/step - loss: 0.4728 - mae: 0.2941 - val_loss: 0.4912 - val_mae: 0.3051 - learning_rate: 0.0010
Restoring model weights from the end of the best epoch: 199.

```

Training History Visualization

Let's visualize how the model performed during training:

```

In [5]: plt.figure(figsize=(12, 4))

# Loss plot
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss Over Time')
plt.xlabel('Epoch')

```



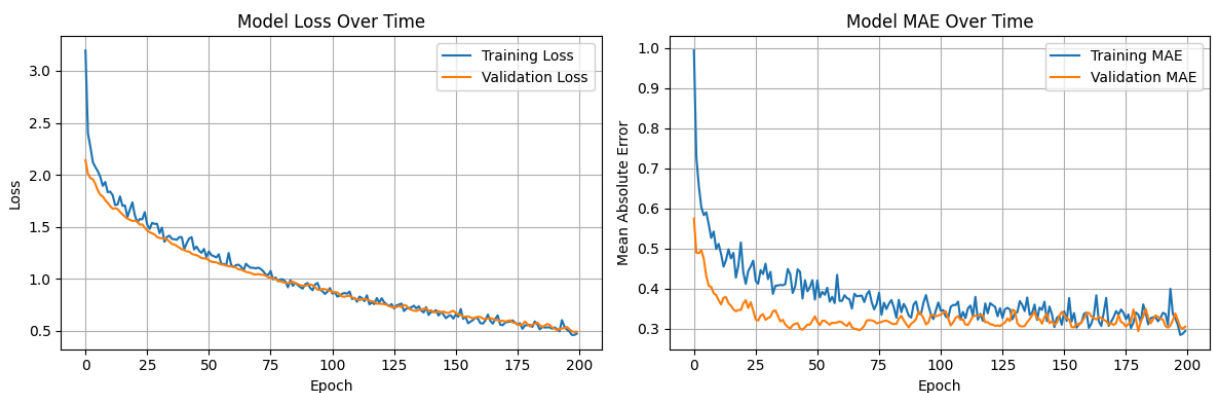
```

plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# MAE plot
plt.subplot(1, 2, 2)
plt.plot(history.history['mae'], label='Training MAE')
plt.plot(history.history['val_mae'], label='Validation MAE')
plt.title('Model MAE Over Time')
plt.xlabel('Epoch')
plt.ylabel('Mean Absolute Error')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

```



Model Evaluation and Predictions

Let's evaluate the model on the test set and make some sample predictions:

```

In [6]: # Evaluate on test set
test_results = model.evaluate(test_dataset, verbose=0)
print(f"Test Mean Absolute Error (normalized): {test_results[1]:.4f}")

# Make predictions
test_predictions = model.predict(X_test)

# Convert predictions back to original scale
test_predictions_original = scaler_y.inverse_transform(test_predictions)
test_actual_original = scaler_y.inverse_transform(y_test)

# Calculate metrics in original scale
mae_original = np.mean(np.abs(test_predictions_original - test_actual_original))
mse = np.mean((test_predictions_original - test_actual_original) ** 2)
rmse = np.sqrt(mse)
r2 = 1 - (np.sum((test_actual_original - test_predictions_original) ** 2) /
          np.sum((test_actual_original - np.mean(test_actual_original)) ** 2))

print(f"\nTest Mean Absolute Error: ${mae_original:.2f}k")
print(f"Root Mean Square Error: ${rmse:.2f}k")
print(f"R-squared Score: {r2:.4f}")

```

```
# Show sample predictions
print("\nSample Predictions vs Actual Values (in $1000s):")
for pred, actual in zip(test_predictions_original[:5], test_actual_original[:5]):
    print(f"Predicted: ${pred[0]:.2f}k, Actual: ${actual[0]:.2f}k")
```

Test Mean Absolute Error (normalized): 0.2430

4/4  0s 36ms/step

Test Mean Absolute Error: \$2.23k

Root Mean Square Error: \$3.28k

R-squared Score: 0.8536

Sample Predictions vs Actual Values (in \$1000s):

Predicted: \$26.56k, Actual: \$23.60k

Predicted: \$34.87k, Actual: \$32.40k

Predicted: \$16.19k, Actual: \$13.60k

Predicted: \$22.31k, Actual: \$22.80k

Predicted: \$16.38k, Actual: \$16.10k

Prediction vs Actual Plot

Let's visualize how well our predictions match the actual values:

```
In [7]: plt.figure(figsize=(10, 6))
plt.scatter(test_actual_original, test_predictions_original, alpha=0.5)
plt.plot([test_actual_original.min(), test_actual_original.max()],
         [test_actual_original.min(), test_actual_original.max()],
         'r--', lw=2)
plt.xlabel('Actual Price ($1000s)')
plt.ylabel('Predicted Price ($1000s)')
plt.title('Predicted vs Actual House Prices')
plt.grid(True)
plt.show()
```

Predicted vs Actual House Prices

